Java Order of Operations

Priority	Operator(s)	Description
1	`( )`	Parentheses
2	`**` , `Math.pow()`	Exponentiation
3	`!`	Logical NOT
4	`*`, `/`, `%`	Multiplication, Division, Modulus
5	`+`, `-`	Addition, Subtraction
6	`>`, `<`, `>=`, `<=`	Relational / Comparison
7	`==`, `!=`	Equality
8	`&&`	Logical AND
9	`||`	Logical OR
10	`=`, `+=`, `-=`	Assignment

Java Control Flow

if

```java
if (condition) {
    // code block
}
```

if/else

```java
if (condition) {
    // code block
} else {
    // code block
}
```

Ternary if/else

```java
variable = (condition) ? valueIfTrue : valueIfFalse;
```

if/else-if

```java
if (condition1) {
    // code block
} else if (condition2) {
    // code block
} else {
    // code block
}
```

Switch

```java
switch(expression) {
    case x:
        // code block
        break;
    case y:
        // code block
        break;
    default:
        // code block
}
```

for loop

```java
for (int i = 0; i < 5; i++) {
    // code block
}
```

while loop

```java
while (condition) {
    // code block
}
```

do-while loop

```java
do {
    // code block
} while (condition);
```

Java Methods

```java
public static <returnType> <methodName>(<parameters>) {
    // code that runs when the method is called
    return <someValue> // Note: Don't need 'return' if <returnType> is `void`
}
```

Java Arrays

Create with values

```
int[] numbers = {10, 20, 30, 40, 50};
String[] colors = {"red", "green", "blue"};
```

Create empty (fixed size)

```
int[] numbers = new int[5];
```

Access / Modify

```
numbers[0]       // first element
numbers[2] = 99; // change third element
numbers.length   // number of elements
```

for-each loop

```
for (int num : numbers) {
    // code block
}
```

Multidimensional Arrays (2D)

Create with values

```
int[][] grid = {
    {1, 2, 3},
    {4, 5, 6}
};
```

Create empty (fixed size)

```
int[][] grid = new int[2][3]; // 2 rows, 3 columns
```

Access / Modify

```
grid[0][2]       // row 0, column 2
grid[1][0] = 99; // change row 1, column 0
grid.length      // number of rows
grid[0].length   // number of columns
```

Nested loop

```
for (int r = 0; r < grid.length; r++) {
    for (int c = 0; c < grid[r].length; c++) {
        System.out.print(grid[r][c] + " ");
    }
    System.out.println();
}
```

ArrayList

Create

```
ArrayList<String> names = new ArrayList<String>();
```

Note: Use object types (`String` , `Integer` , `Double`), not primitives.

Common methods

```
names.add("Alice");      // add to end
names.get(0);            // access by index
names.set(0, "Bob");     // change element
names.remove("Alice");   // remove by value
names.size();            // number of elements
names.contains("Bob");   // check if exists
```